

Working Memory Layer

Agentic AI Architecture — Complete Engineering Reference

Gaurav Kumar · June 11, 2026 · v1.0

This document covers the full working memory layer for an agentic AI system — from the rolling-buffer architecture and async compression pipeline to token budget enforcement, summary drift prevention, and the anchored compression prompt. The episodic, semantic, and procedural layers are covered in separate documents.

#	Section	Key Concept
1	What Working Memory Is	Rolling buffer · within-session only
2	Mathematical Framework	T, R, C variables · preset modes
3	Full Architecture	Async fork · SSE control frame · silent update
4	Compression Pipeline	LLM extraction · anchored prompt
5	Summary Drift Prevention	Anchored compression · grounding pass
6	Token Budget Enforcement	Pre-flight check · tiktoken · priority order
7	Compression Prompt (Full)	Entity extraction · verbatim preservation
8	Edge Cases & Fallbacks	Slow LLM · short responses · failures
9	User Config & Presets	Precise / Balanced / Turbo
10	Ratings & Next Steps	Honest assessment · what to build next

Definition

Working memory is Layer 1 of the full memory stack. It is the context the LLM sees on every single turn — assembled fresh each time a message is sent. It has no persistence across sessions. When the tab closes, it is gone. Everything else (episodic, semantic, procedural layers) builds on top of this.

What it contains (assembled each turn)

- System prompt + user profile constraints — always present, non-negotiable
- Rolling summary — compressed history of old messages that no longer fit in raw form
- Raw buffer (R) — the most recent R messages in full, uncompressed
- Current user message

What it is NOT

- Not a database — nothing here survives a session end
- Not episodic memory — it does not retrieve past sessions
- Not the same as LangMem storage — LangMem handles layers 2–4

The single most important property: the LLM must never see more tokens than its context window allows. Working memory is the layer that enforces this constraint in real time.

2 — Mathematical Framework

Core Variables

Variable	Name	Description	Default
T	Trigger Threshold	Total messages before compression fires	30
R	Raw Buffer	Recent messages kept uncompressed	10
C	Chunk Size	Messages sent for compression ($C = T - R$)	20

Constraints

- Hard safety: $T > R \geq 1$
- Efficiency: $T - R \geq 5$ (compressing fewer than 5 messages is wasteful)
- Recommended: $T - R \geq 10$ for balanced performance

Preset Modes

Mode	T	R	C	Best For
Precise	50	40	10	High accuracy, minimal compression, higher token cost
Balanced	30	10	20	Default — good cost/quality trade-off
Turbo	15	5	10	Low cost, aggressive compression, shorter sessions

Recommendation: hardcode Balanced mode to start. Do not expose T/R sliders to users until you have evidence they need it. Ship first, configure later.

Five-Phase Flow

Every user message travels through five phases. Phases 1 and 5 are synchronous and visible to the user. Phases 2–4 happen in parallel, invisibly.

Phase 1	Frontend → Backend	React checks if <code>len(messages) >= T</code> . Sends payload: <code>rolling_summary</code> + last <code>R</code> messages + new message. Does NOT send full history — only the window.
Phase 2	Async Fork (Backend)	If compression is needed, backend spawns two concurrent tasks. Task A: call primary LLM with full current payload → stream response to user. Task B: call Memory LLM with oldest <code>C</code> messages + current <code>rolling_summary</code> → produce updated summary. Tasks run in parallel via <code>asyncio.gather()</code> .
Phase 3	SSE Dual-Lane Stream	Lane 1 streams text tokens to the user's chat bubble as usual. Lane 2 is a heartbeat that monitors Task B. When Task B completes, backend injects a control frame: <code>{control: memory_compact, rolling_summary: ..., truncated_count: C}</code> . This arrives mid-stream or after the last token.
Phase 4	Silent Frontend Update	React intercepts the control frame. Updates <code>rolling_summary</code> in state. Slices messages array: <code>messages.slice(truncated_count)</code> . The UI still shows all messages in the scrollable history — only the API payload shrinks. Zero flicker, zero user-visible change.
Phase 5	Clean State	Frontend now holds: new <code>rolling_summary</code> + only <code>R</code> raw messages. Next user message automatically uses the lean payload. Token cost is bounded regardless of conversation length.

4 — Compression Pipeline

What the Memory LLM does

The Memory LLM is a separate, cheaper LLM call (can use `claude-haiku` or `gpt-4o-mini`). It receives the oldest `C` messages plus the existing rolling summary, and produces an updated summary. This is not a generic summarisation task — it is a structured extraction job.

What must be preserved

- All unique identifiers: file paths, variable names, function names, user names
- All decisions made: 'User chose Python over Java at turn 7'
- All constraints stated: 'User wants responses in Hindi'
- Bugs fixed and their locations
- Any explicit user preferences mentioned

What must NOT appear in the summary

- Generic filler: 'The user and assistant discussed several topics'
- Paraphrased versions of facts already in the existing summary
- Opinions or interpretations not stated by the user

The Problem

Summary drift is a silent failure mode. Each compression round, the Memory LLM slightly rephrases the existing summary. Over 5–6 rounds, facts get distorted, names change subtly, and decisions get lost. The user notices the AI 'forgetting' things it was told hours ago.

Root cause: the LLM treats the existing summary as input to rewrite, not as a contract to preserve.

Solution 1 — Anchored Compression Prompt (primary fix)

Change the compression instruction from 'summarise this' to 'extend this'. The prompt explicitly instructs the LLM to treat every fact in the existing summary as immutable unless a newer message directly contradicts it.

```
COMPRESSION PROMPT (Anchored): You are a memory manager. Your job is NOT to rewrite the existing
summary. Your job is to EXTEND it with new information from the conversation chunk. Rules: 1.
Preserve every fact, name, path, variable, and decision from the existing summary EXACTLY as
written. 2. Only add new information from the conversation chunk below. 3. If new information
contradicts an existing fact, update only that specific fact and mark it: [UPDATED: old value →
new value]. 4. Do NOT paraphrase existing content. Do NOT restructure. 5. Output format: [EXISTING
- preserved verbatim] ...existing summary here... [NEW - from this chunk] ...only new facts
here... Existing Summary: {rolling_summary} Conversation Chunk (messages {start} to {end}):
{chunk_messages}
```

Solution 2 — Grounding Pass (secondary, run every 5th compression)

After every 5th compression cycle, run a separate grounding LLM call. Send the last 3 rolling summaries and ask the LLM to reconcile them into one canonical version. This resets accumulated micro-distortions before they become user-visible.

```
GROUNDING PROMPT: You are given 3 versions of a conversation summary from different points in time
(oldest to newest). Produce one canonical summary that: - Keeps the most recent version of any
fact that changed - Removes any contradiction between versions - Preserves ALL unique identifiers,
decisions, and constraints - Does not add any new information Summary v{n-2}: {old_summary}
Summary v{n-1}: {mid_summary} Summary v{n}: {latest_summary}
```

Why the Raw Buffer also helps

The raw buffer (R) acts as a natural drift guard for recent context. Even if the summary has minor drift, the last R messages are always present verbatim — so the LLM always has accurate recent context. Drift in the summary only affects older history, which matters less for the immediate conversation.

The Problem

Message count (T, R) is a proxy for token count — and a bad one. A single message containing a pasted code block can be 800 tokens. A casual reply is 12 tokens. Relying on message count alone means you will eventually exceed the model's context window and get a hard API error.

Pre-flight Token Check (run before every LLM call)

```
import tiktoken
enc = tiktoken.encoding_for_model("gpt-4o")
def build_payload(system_prompt, rolling_summary, messages, model_limit=128000, output_reserve=2000):
    budget = model_limit - output_reserve
    # always reserve room for output
    # Non-negotiable: always include these
```

```
system_tokens = len(enc.encode(system_prompt)) summary_tokens = len(enc.encode(rolling_summary))
used = system_tokens + summary_tokens if used > budget: # Summary itself is too large – truncate
it (emergency only) raise ValueError("Summary exceeds budget – trigger grounding pass") # Add
messages newest-first until budget runs out messages_to_send = [] for msg in reversed(messages):
cost = len(enc.encode(msg["content"])) + 4 # 4 = message overhead if used + cost > budget: break #
stop here, don't include older messages messages_to_send.insert(0, msg) used += cost return {
"system": system_prompt, "rolling_summary": rolling_summary, "messages": messages_to_send,
"tokens_used": used, "tokens_remaining": budget - used, }
```

Priority order — what gets included when budget is tight

Priority	Content	Drop when tight?
1 — Never drop	System prompt + user constraints	No — always included
2 — Never drop	Rolling summary	No — more dense than raw messages
3 — Never drop	Last 5 messages	No — minimum coherence floor
4 — Drop last	Messages 6 to R	Yes — drop oldest first
5 — Not in payload	Messages older than R	Already compressed into summary

Handling oversized single messages

- If one message > 30% of remaining budget: summarise it individually before storing in the messages array
- Never silently drop a message — append [truncated — {N} tokens removed] so the LLM knows context is incomplete
- Code blocks are the most common culprit — consider storing them as file references rather than inline content

7 — Compression Prompt (Production Version)

This is the full production prompt for the Memory LLM. It combines anchored compression (no drift) with structured entity extraction (no information loss). Tune the entity types to match your agent's domain.

```
SYSTEM: You are a memory compression specialist for an AI assistant. You do not summarise – you
EXTEND and PRESERVE. USER: == EXISTING SUMMARY (treat as immutable contract) == {rolling_summary}
== NEW CONVERSATION CHUNK (messages {start_idx} to {end_idx}) == {chunk_messages} == YOUR TASK ==
1. PRESERVE: Copy the existing summary below, word for word. 2. EXTEND: Below the existing
summary, add only NEW facts from the chunk. 3. UPDATE: If a new message directly contradicts an
existing fact, update only that specific line and mark it: [UPDATED: {old} → {new}] 4. EXTRACT:
List all new entities found in the chunk. == OUTPUT FORMAT == --- PRESERVED SUMMARY --- [copy
existing summary here, verbatim] --- NEW ADDITIONS --- [only new facts, decisions, preferences,
constraints] --- ENTITIES (new in this chunk) --- Files/Paths: [list] Variables/Functions: [list]
Decisions made: [list] User preferences stated: [list] Errors/Bugs: [list] Other identifiers:
[list] == RULES == - DO NOT paraphrase the preserved section - DO NOT restructure or reorder - DO
NOT add generic filler sentences - If nothing new was found, write "No new additions" in the NEW
ADDITIONS section - Be specific: "User chose FastAPI over Flask for the auth service" NOT "User
made a technology choice"
```

8 — Edge Cases and Fallbacks

Scenario	What Happens	Solution
Compression LLM is slow	User sends next message before summary is ready	LLM already uses raw history for that turn. Injects new summary on the turn after it
Compression LLM fails / errors	No updated summary produced	Continue with existing summary + raw history. Log the failure. Retry compression
User sends a very long message	Single message exceeds budget	Summarise the message individually before adding to the messages array. Stop
Summary itself grows too large	After many grounding passes, summary bloats	Cap summary at 800 tokens. If grounding pass output exceeds cap, instruct LLM to
Very short assistant reply	Compression finishes after the reply — context buffer is empty	Fill context buffer with a control frame and applies it before the next user message

9 — User Configuration

Recommendation: hardcode Balanced for V1

Do not build the settings panel until users ask for it. The vast majority of users will never tune T and R. Balanced mode (T=30, R=10) works well for 95% of conversations. Add config only when you have a specific user request that Balanced cannot serve.

If you do build the settings panel — validation rules

- Hard block: T must be > R. If user drags R above T, auto-raise T.
- Hard block: T – R must be ≥ 5. Show error if violated.
- Soft warning: T – R < 10 → 'Compressing frequently — may reduce coherence'
- Soft warning: R < 5 → 'Low raw buffer — recent context may be thin'

Mode	T	R	C	Token Cost	Coherence	When to Use
Precise	50	40	10	High	Excellent	Long technical sessions, code review
Balanced	30	10	20	Medium	Good	Default for all use cases
Turbo	15	5	10	Low	Acceptable	Short Q&A, simple tasks, cost-sensitive

Honest Rating: 8.5 / 10

With the drift prevention, token budget enforcement, and anchored compression prompt added, this is a production-grade working memory layer. The async fork architecture is genuinely strong engineering. The main remaining risk is compression prompt quality — bad prompts cause slow, invisible degradation.

Component	Rating	Notes
Async fork architecture	10/10	Best design decision. Zero-latency. Production-ready.
SSE control frame pattern	9/10	Clean. Invisible to user. Easy to debug.
Rolling buffer math (T/R/C)	9/10	Explicit, testable, predictable.
Fallback chain	8/10	Covers the important cases. Add retry logic later.
Compression prompt	7/10	Good with anchored version. Needs domain tuning.
Token budget enforcement	7/10	Solid with tiktoken pre-flight. Test with code-heavy sessions.
Drift prevention	8/10	Anchored prompt solves 80%. Grounding pass solves the rest.
User config / presets	6/10	Over-engineered for V1. Simplify until needed.

Build Order — what to implement in what sequence

Sprint	Task	Why Now
Day 2 (now)	Rolling 20-message window in React state	Foundation — nothing works without this
Day 2 (now)	Send full messages array to backend each turn	Fixes the broken memory feel immediately
Week 1	Async compression fork + SSE control frame	Bounds token cost for long conversations
Week 1	Anchored compression prompt	Without this, drift starts at compression #3
Week 1	tiktoken pre-flight budget check	Prevents hard API errors from long code blocks
Week 2	Grounding pass (every 5th compression)	Resets drift that slips through the prompt
Week 2	Oversized message handler	Edge case but hits every technical user
Week 3+	User config panel (if needed)	Only if users request it — don't build speculatively

Next document: Episodic Memory Layer — storing, retrieving, and injecting past session summaries across conversations.